POO + AUTOMATIZACIÓN · LABORATORIO INTEGRADOR

# Laboratorio #1 — Extensión

Para quienes terminen antes de tiempo

| MODALIDAD | REQUISITO PREVIO | DURACIÓN SUGERIDA | NIVEL |
|---|---|---|---|
| **Individual** | **Laboratorio #1 funcionando** | **Sin límite fijo** | **Retador** |

Esta extensión no es obligatoria — está pensada para quienes terminen el Laboratorio #1 con tiempo de sobra y quieran llevar su solución un paso más allá. Elige uno o varios de los siguientes retos; cada uno se apoya en cosas que ya sabes hacer, aplicadas a un contexto nuevo.

> **Antes de empezar la extensión:** tu Laboratorio #1 base debe estar completamente funcional. No avances aquí a costa de dejar la solución principal a medias.

## Reto 1: Jerarquía de clases con herencia y polimorfismo

Amplía tu modelo de clases: crea una clase base y al menos dos clases hijas que hereden de ella y sobreescriban un mismo método, cada una a su manera (polimorfismo real, no solo clases sueltas). Por ejemplo, si tu escenario tiene proveedores, crea una jerarquía `Proveedor` → `ProveedorNacional` / `ProveedorInternacional`, cada uno calculando el tiempo de entrega o el nivel de riesgo de forma distinta. Tu nodo `Code` en n8n debe reflejar esta misma jerarquía en su lógica.

## Reto 2: Segunda fuente externa + Merge

Conecta una **segunda** API externa a tu workflow (distinta a la que ya usaste), y utiliza un nodo `Merge` para combinar ambas fuentes antes de tomar tu decisión final. Por ejemplo: si tu escenario ya usa el tipo de cambio del BCCR, agrega también los feriados de `date.nager.at` para no procesar pagos en días no hábiles.

## Reto 3: Notificación automática por correo

Si tu escenario todavía no envía correos, agrega un nodo de envío de correo al final de tu workflow para notificar automáticamente cuando se detecte una situación crítica (ej. un proveedor moroso, un producto agotado, o un tipo de cambio disparado). Si tu escenario ya recibía correos como entrada (Escenario 3), en vez de esto agrega una respuesta automática de confirmación al remitente original.

## Reto 4: Historial y tendencia

En vez de reportar solo el estado actual, guarda un historial de las últimas ejecuciones del workflow (puedes agregarlo como una colección más en JSON Server) y crea una clase adicional (ej. `HistorialTendencia`) con un método que determine, a partir de ese historial, si la situación está mejorando o empeorando con el tiempo — y muestra esa conclusión en el resultado final del workflow.

> **Recomendación:** no intentes los 4 retos a la vez. Elige el que se conecte mejor con tu escenario de negocio — la extensión vale más si se siente parte natural de tu solución, no un agregado forzado.